

CRM RADIO MÓVILES

FASE 1: Base y Autenticación

Documentación Técnica

Generado: 19 de May, 2026

Tarija, Bolivia

Este documento detalla la construcción de la **Fase 1** del CRM para Plus Móvil. Cubre: configuración del entorno Docker, base de datos Supabase, autenticación Firebase, y estructura fundamental del sistema con los 3 roles de usuario.

TABLA DE CONTENIDOS

1. Introducción
2. Arquitectura de Fase 1
3. Setup del Entorno
4. Estructura de Carpetas
5. Variables de Entorno
6. Base de Datos (Supabase)
7. Autenticación (Firebase)
8. Backend (Node.js + Express)
9. Frontend (React + Vite)
10. Login Component
11. Próximos Pasos

1. INTRODUCCIÓN

La **Fase 1** establece los cimientos del CRM. Su objetivo es tener un sistema funcional que permita a los usuarios autenticarse según su rol (Operadora, Contadora, Admin) y acceder a sus vistas correspondientes.

Al final de esta fase tendrás:

- ✓ Proyecto corriendo en Docker
- ✓ Base de datos Supabase conectada
- ✓ Login con Firebase Auth y 3 roles funcionando
- ✓ Panel de bienvenida según el rol del usuario

2. ARQUITECTURA DE FASE 1

Frontend: React + Vite (Puerto 5173)

Backend: Node.js + Express (Puerto 3000)

Base de Datos: Supabase PostgreSQL

Autenticación: Firebase Auth

Containerización: Docker Compose

Flujo de autenticación:

1. Usuario ingresa email y contraseña en el login
2. Firebase Auth valida las credenciales
3. Backend verifica el rol en Supabase
4. Frontend redirige a la vista correspondiente (Operadora / Contadora / Admin)

3. SETUP DEL ENTORNO

Requisitos previos:

- Docker Desktop instalado
- Node.js 18+ instalado
- Git
- Cuenta en Supabase (gratuita)
- Proyecto en Firebase Console

Paso 1: Crear el repositorio

```
mkdir crm-radio-moviles
cd crm-radio-moviles
git init
```

Paso 2: Estructura de carpetas

```
crm-radio-moviles/
├── backend/ (Node.js + Express)
├── frontend/ (React + Vite)
├── docker-compose.yml
├── .env.example
└── README.md
```

4. ESTRUCTURA DE CARPETAS DETALLADA

Backend:

```
backend/  
  ■■■ src/  
  ■ ■■■ config/ (Conexión a Supabase, Firebase)  
  ■ ■■■ routes/ (API endpoints)  
  ■ ■■■ controllers/ (Lógica de negocio)  
  ■ ■■■ middleware/ (Auth, logging)  
  ■ ■■■ index.js (Entry point)  
  ■■■ .env  
  ■■■ package.json  
  ■■■ Dockerfile  
  ■■■ .dockerignore
```

Frontend:

```
frontend/  
  ■■■ src/  
  ■ ■■■ components/ (Componentes React)  
  ■ ■■■ pages/ (Vistas principales)  
  ■ ■■■ hooks/ (Custom hooks)  
  ■ ■■■ utils/ (Funciones auxiliares)  
  ■ ■■■ App.jsx  
  ■ ■■■ main.jsx  
  ■■■ .env  
  ■■■ package.json  
  ■■■ vite.config.js  
  ■■■ Dockerfile
```

5. VARIABLES DE ENTORNO

.env (backend)

```
PORT=3000
NODE_ENV=development

# Supabase
SUPABASE_URL=https://xxxxx.supabase.co
SUPABASE_ANON_KEY=xxxxx
SUPABASE_SERVICE_KEY=xxxxx

# Firebase
FIREBASE_PROJECT_ID=xxxxx
FIREBASE_PRIVATE_KEY=xxxxx
FIREBASE_CLIENT_EMAIL=xxxxx

CORS_ORIGIN=http://localhost:5173
```

.env (frontend)

```
VITE_API_URL=http://localhost:3000

# Firebase Config
VITE_FIREBASE_API_KEY=xxxxx
VITE_FIREBASE_AUTH_DOMAIN=xxxxx.firebaseio.com
VITE_FIREBASE_PROJECT_ID=xxxxx
VITE_FIREBASE_STORAGE_BUCKET=xxxxx.appspot.com
VITE_FIREBASE_MESSAGING_SENDER_ID=xxxxx
VITE_FIREBASE_APP_ID=xxxxx
```

6. BASE DE DATOS (SUPABASE)

Paso 1: Crear proyecto en supabase.com (gratis)

Paso 2: Copiar URL y API keys a tu .env backend

Paso 3: Crear tablas usando SQL en el dashboard de Supabase

SQL para crear tablas (en Supabase):

```
-- Tabla de usuarios (roles)
CREATE TABLE users_roles (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  email TEXT UNIQUE NOT NULL,
  role TEXT CHECK (role IN ('operadora', 'contadora', 'admin')),
  created_at TIMESTAMP DEFAULT NOW()
);

-- Tabla de operadoras (para datos de turnos)
CREATE TABLE operadoras (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id UUID REFERENCES users_roles(id),
  nombre TEXT NOT NULL,
  fecha_ingreso TIMESTAMP DEFAULT NOW()
);
```

7. AUTENTICACIÓN (FIREBASE)

Paso 1: Ir a firebase.google.com → Create Project

Paso 2: Habilitar Authentication → Email/Password

Paso 3: Crear usuarios de prueba en Firebase Console:

- operadora@test.com (contraseña: Test123!)
- contadora@test.com (contraseña: Test123!)
- admin@test.com (contraseña: Test123!)

Paso 4: Descargar credenciales JSON (Service Account) y guardar en backend

8. BACKEND (NODE.JS + EXPRESS)

Crear proyecto backend:

```
mkdir backend
cd backend
npm init -y
npm install express cors dotenv @supabase/supabase-js firebase-admin
```

backend/src/index.js

```
const express = require('express');
const cors = require('cors');
require('dotenv').config();
const { createClient } = require('@supabase/supabase-js');

const app = express();
app.use(cors({ origin: process.env.CORS_ORIGIN }));
app.use(express.json());

const supabase = createClient(
  process.env.SUPABASE_URL,
  process.env.SUPABASE_ANON_KEY
);

app.post('/api/auth/verify', async (req, res) => {
  const { uid, email } = req.body;
  const { data } = await supabase
    .from('users_roles')
    .select('role')
    .eq('email', email)
    .single();
  res.json({ role: data?.role || 'guest' });
});

app.listen(process.env.PORT || 3000);
```

9. FRONTEND (REACT + VITE)

Crear proyecto frontend:

```
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm install firebase react-router-dom
```

frontend/src/main.jsx

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'
import App from './App'
import './App.css'

ReactDOM.createRoot(document.getElementById('root')).render(

  ,
)
```

10. LOGIN COMPONENT

frontend/src/components/Login.jsx

```
import React, { useState } from 'react'
import { signInWithEmailAndPassword } from 'firebase/auth'
import { auth } from '../config/firebase'
import { useNavigate } from 'react-router-dom'

export default function Login() {
  const [email, setEmail] = useState('')
  const [password, setPassword] = useState('')
  const [error, setError] = useState('')
  const navigate = useNavigate()

  const handleLogin = async (e) => {
    e.preventDefault()
    try {
      const result = await signInWithEmailAndPassword(auth, email, password)
      const token = await result.user.getIdToken()
      const response = await fetch('http://localhost:3000/api/auth/verify', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ uid: result.user.uid, email })
      })
      const { role } = await response.json()
      localStorage.setItem('role', role)
      navigate(`/dashboard/${role}`)
    } catch (err) {
      setError(err.message)
    }
  }

  return (
    setEmail(e.target.value)} />
    setPassword(e.target.value)} />
    Login
    {error && {error}}
  )
}
```

11. DOCKER COMPOSE

docker-compose.yml (raíz del proyecto)

```
version: '3.8'
services:
  backend:
    build: ./backend
    ports:
      - '3000:3000'
    environment:
      - NODE_ENV=development
      - SUPABASE_URL=${SUPABASE_URL}
      - SUPABASE_ANON_KEY=${SUPABASE_ANON_KEY}
    volumes:
      - ./backend/src:/app/src

  frontend:
    build: ./frontend
    ports:
      - '5173:5173'
    environment:
      - VITE_API_URL=http://localhost:3000
    volumes:
      - ./frontend/src:/app/src
```

Para correr todo:

```
docker-compose up --build
```

12. PRÓXIMOS PASOS

✓ Fase 1 completada: Login y roles funcionando

→ **Fase 2:** WhatsApp gateway, panel operadora, gestión de turno

→ **Fase 3:** Fichas de choferes, módulo contadora

→ **Fase 4:** Dashboard analítico, mapa, deploy en Render

Contacto: Koffy (Kevin Flores) | Tarija, Bolivia

Fecha: 19 de May, 2026